

UNIVERSIDAD INTERNACIONAL DEL ECUADOR



Escuela de Ingeniería en Ciberseguridad

Programación Orientada a Objetos 2

Aprendizaje Autónomo 1

Tema: Selección de sistema de
Gestión Empresarial

Jorge Cano

Quito, 21 de Julio, 2026

Link de acceso al repositorio en Github:

<https://github.com/JCcoconut/sistema-ecommerce-go>

Sistema de gestión de e-commerce en Go

1. Introducción

El presente proyecto propone el desarrollo de un sistema de gestión de comercio electrónico denominado AudioCyber Store, orientado a la venta simulada de productos tecnológicos, periféricos, accesorios y equipos de audio.

La aplicación será desarrollada en lenguaje Go y funcionará mediante una interfaz de consola.

Aunque la asignatura corresponde a programación orientada a objetos, la consigna establece que el software debe desarrollarse mediante programación funcional. Por esta razón, las estructuras de Go se utilizarán principalmente como contenedores de datos, mientras que las reglas del negocio se implementarán mediante funciones independientes.

Objetivo del sistema

2.1 Objetivo general

Desarrollar un sistema de gestión de e-commerce en Go que permita administrar productos, carrito de compras, promociones, inventario y pedidos, aplicando programación funcional y los fundamentos estudiados en la Unidad 1.

2.2 Objetivos específicos

- Diseñar una estructura modular que separe las responsabilidades del sistema.
- Registrar y consultar los productos disponibles.
- Permitir la búsqueda de productos por nombre, categoría o identificador.
- Gestionar los productos y cantidades agregados al carrito.
- Calcular subtotal, descuentos y valor final de una compra.
- Validar la disponibilidad de inventario antes de confirmar un pedido.
- Generar pedidos con identificadores únicos.
- Utilizar funciones como valores y argumentos de otras funciones.
- Emplear arrays, slices y maps según las necesidades de cada módulo.
- Organizar el código en paquetes propios de Go.
- Documentar los paquetes de la biblioteca estándar y cualquier dependencia externa.

Alcance del proyecto

3.1 Funcionalidades incluidas

Módulo	Responsabilidad	Operaciones incluidas
Catálogo	Presentar la oferta disponible.	Listar, ordenar y buscar por nombre o categoría.
Carrito	Construir la intención de compra.	Agregar, acumular cantidades, retirar, vaciar y calcular unidades.
Promociones	Aplicar reglas variables al total.	Validar cupones y construir descuentos de 10 %, 15 % o 20 %.
Pedidos	Confirmar y conservar compras.	Calcular totales, seleccionar pago simulado, generar ID e historial.
Inventario	Mantener existencias coherentes.	Validar stock, descontar de forma atómica y reponer unidades.

El prototipo permitirá:

- Mostrar el catálogo de productos.
- Ordenar los productos alfabéticamente.
- Buscar productos por nombre o categoría.
- Consultar precio y disponibilidad.
- Agregar productos al carrito.
- Acumular cantidades de un mismo producto.
- Retirar productos del carrito.
- Vaciar el carrito.
- Calcular el número total de unidades.
- Calcular subtotal, descuento y total.
- Aplicar cupones promocionales.
- Seleccionar un método de pago simulado.
- Confirmar una compra.
- Generar un identificador para cada pedido.
- Descontar las cantidades compradas del inventario.
- Consultar el historial de pedidos.
- Reponer existencias de un producto.

3.2 Funcionalidades fuera del alcance

En esta primera versión no se desarrollarán:

- Interfaz web o aplicación móvil.
- Registro y autenticación de usuarios.
- Base de datos.
- Pagos reales.
- Almacenamiento de tarjetas.
- Cálculo de impuestos.
- Gestión de direcciones y envíos.

- Comunicación con empresas de transporte.
- Facturación electrónica.
- Persistencia de información después de cerrar el programa.
- Procesamiento concurrente de pedidos.

Estas funcionalidades podrían incorporarse en futuras unidades.

Arquitectura y estructura del sistema

El paquete main conserva el estado de la sesión y coordina las funciones de dominio; los demás paquetes reciben valores y retornan resultados. Esta división evita que la lógica dependa directamente de la consola.

Nivel	Componentes	Función arquitectónica
Presentación	cmd/tienda	Menú, lectura de datos, mensajes y coordinación del flujo.
Dominio	catalogo · carrito · pedido	Búsqueda, inventario, carrito, promociones, totales y confirmación.
Datos compartidos	modelo	Estructuras Producto, ItemCarrito, LineaPedido y Pedido.

4.1 Flujo principal

Paso	Entrada	Transformación	Salida
1	Catálogo inicial	Buscar u ordenar productos	Productos visibles
2	Producto + cantidad	Validar existencia y stock	Nuevo carrito
3	Cupón	Crear función de descuento	Regla activa
4	Carrito + regla	Calcular subtotal, descuento y total	Resumen de compra
5	Método de pago	Validar opción fija	Pago simulado seleccionado
6	Pedido confirmado	Descontar inventario de forma atómica	Pedido e inventario nuevos

4.2 Estructura de archivos

```
ecommerce-go/
├── go.mod
├── README.md
├── cmd/tienda/main.go
├── internal/
│   ├── modelo/modelo.go
│   ├── catalogo/catalogo.go
│   ├── carrito/carrito.go
│   └── pedido/pedido.go
```

Paquetes incorporados en el sistema

1. Propios

Paquete	Responsabilidad	Funciones representativas
modelo	Tipos compartidos del dominio.	Producto, ItemCarrito, LineaPedido y Pedido.
catalogo	Consulta y transformación del inventario.	Inicial, BuscarPorID, BuscarPorTexto, DescontarStock.
carrito	Construcción y cálculo del carrito.	Agregar, Eliminar, CantidadTotal, Subtotal.
pedido	Promociones, totales, pagos e identificación.	CrearDescuento, CalcularTotales, NuevoGeneradorID, Crear.
main	Interfaz y composición de casos de uso.	mostrarMenu, leerEntero, mostrarCarrito, mostrarPedidos.

Módulo modelos:

Producto(), ItemCarrito(), LineaPedido(), Pedido()

Módulo catálogo:

CatalogoInicial(), BuscarPorID(), BuscarPorTexto(), OrdenarPorNombre(), DescontarStock(), ReponerStock()

Funcionalidades

- Crear el catálogo inicial.
- Mostrar todos los productos.
- Buscar un producto por ID.
- Buscar productos por nombre.
- Filtrar productos por categoría.
- Ordenar el catálogo.
- Crear una copia del catálogo antes de modificarlo.

Módulo carrito:

AgregarProducto(), EliminarProducto(), VaciarCarrito(), CantidadTotal(), CalcularSubtotal()

Reglas principales

- La cantidad debe ser mayor que cero.
- La cantidad acumulada no puede superar el stock.
- Si se produce un error, el carrito original no debe modificarse.
- El subtotal se obtiene multiplicando el precio por la cantidad de cada producto.

Módulo pedido:

CrearDescuento(), CalcularTotales(), CrearPedido(), NuevoGeneradorID(),
MetodosPagoDisponibles()

Funcionalidades

- Calcular subtotal.
- Aplicar descuentos.
- Calcular el valor final.
- Generar líneas del pedido.
- Seleccionar un método de pago.
- Generar un identificador único.
- Registrar la compra en el historial.

Ejemplo de cupones:

"AUDIO10": 0.10,
"GO15": 0.15,
"UIDE20": 0.20,

Funcionalidades

- Registrar cupones.
- Validar si un cupón existe.
- Obtener su porcentaje.
- Construir una función de descuento.
- Aplicar la función sobre el subtotal.

Módulo main:

MostrarMenu(), LeerTexto(), LeerEntero(), MostrarProductos(), MostrarCarrito()

Funcionalidades

- Mostrar el menú.
- Leer texto.
- Leer números.
- Validar opciones.
- Mostrar productos.
- Mostrar el carrito.
- Mostrar pedidos.
- Coordinar los módulos.

Punteros mediante **bufio.Reader*. El puntero se utiliza en la frontera de entrada porque se necesita compartir el mismo lector durante toda la ejecución. Las funciones del dominio no dependen de este puntero.

2. Biblioteca estándar

Paquete	Uso en el proyecto
bufio	Lectura eficiente y reutilizable desde la entrada estándar.
errors	Creación de errores de validación como valores retornables.
fmt	Salida formateada, mensajes y construcción de identificadores.
os	Acceso a la entrada estándar del proceso.
sort	Ordenamiento de una copia del catálogo mediante una función literal.
strconv	Conversión segura del texto del usuario a enteros.
strings	Normalización, búsqueda y presentación de cadenas.
time	Registro de la fecha y hora de confirmación del pedido.
testing	Pruebas unitarias de los paquetes de dominio.

Reglas de negocio y validaciones

Regla	Respuesta del sistema
Cantidad menor o igual a cero	Se rechaza y se devuelve un error.
Producto inexistente	La búsqueda devuelve false o la operación devuelve error.
Cantidad superior al stock	El carrito no se modifica.
Cupón inexistente	No se reemplaza la regla activa.
Carrito vacío	No se permite confirmar el pedido.
Método de pago fuera de 1–3	Se rechaza la selección.
Fallo de una línea al descontar stock	No se descuenta ninguna línea; la transformación es atómica.
Pedido confirmado	Se guarda una fotografía de nombre y precio para preservar el historial.

Estrategia de pruebas

Prueba	Propósito	Resultado esperado
TestReponerStockNoModificaOriginal	Validar copia antes de transformar.	Original = 2; resultado = 5.
TestDescontarStockEsAtomico	Evitar descuentos parciales.	Error y catálogo original intacto.
TestAgregarAcumulaCantidadSinModificarOriginal	Acumular cantidades sin efecto lateral.	Original = 1; resultado = 3.
TestCantidadTotalRecursiva	Validar el cálculo recursivo.	$2 + 3 + 1 = 6$.
TestCrearDescuento	Verificar closure GO15.	100 se transforma en 85.
TestGeneradorIDMantieneEstado	Verificar estado privado del closure.	PED-001 y PED-002.
TestSeleccionarMetodoPago	Validar el array de métodos fijos.	2 = Transferencia; 4 se rechaza.

Closures

CrearDescuento retorna una función anónima que captura el porcentaje asociado al cupón. NuevoGeneradorID aplica el mismo principio para conservar un contador privado y producir PED-001, PED-002 y valores sucesivos sin una variable global.

Manejo explícito de errores

Las funciones retornan el resultado y un valor error cuando la operación puede fallar. Por ejemplo, ReponerStock devuelve ([Producto, error], mientras BuscarPorID devuelve (Producto, bool). El programa decide cómo presentar cada resultado sin recurrir a excepciones.

6. Flujo general del sistema

1. El programa crea el catálogo inicial.
2. Se presenta el menú principal.
3. El cliente consulta o busca productos.
4. Selecciona un producto y una cantidad.
5. El módulo del carrito valida la solicitud.
6. El cliente puede retirar productos o aplicar un cupón.
7. El módulo de pedidos calcula subtotal, descuento y total.
8. El usuario selecciona un medio de pago simulado.
9. El módulo de inventario valida todas las existencias.
10. El sistema genera el pedido.
11. Se descuentan las cantidades compradas.
12. El carrito se vacía.
13. El pedido queda disponible en el historial.

7. Limitaciones y evolución propuesta

El prototipo cumple el propósito de la Unidad 1, pero no pretende reemplazar una plataforma comercial. La información desaparece al cerrar el proceso y los valores monetarios se representan con float64 por simplicidad. En un entorno productivo sería preferible trabajar con centavos enteros o una biblioteca decimal para impedir errores de redondeo.

- Persistir productos, clientes y pedidos en una base de datos relacional.
- Incorporar autenticación, autorización y separación de roles de cliente y administrador.
- Exponer la lógica mediante una API HTTP y desarrollar una interfaz web.
- Integrar impuestos, direcciones, logística y estados del pedido.
- Añadir una pasarela real con tokenización; nunca almacenar datos de tarjeta en texto claro.
- Agregar pruebas de integración, concurrencia y análisis de cobertura.

8. Estructura Propuesta

sistema-ecommerce-go/

```
|— .github/
|   └─ workflows/
|       └─ go.yml
|— cmd/
|   └─ tienda/
|       └─ main.go
|— docs/
|   └─ estructura-funcionalidades-alcance.pdf
|— internal/
|   └─ modelo/
|       └─ modelo.go
|   └─ catalogo/
|       └─ catalogo.go
|       └─ catalogo_test.go
|   └─ carrito/
|       └─ carrito.go
|       └─ carrito_test.go
|   └─ promocion/
|       └─ promocion.go
|       └─ promocion_test.go
|   └─ inventario/
|       └─ inventario.go
|       └─ inventario_test.go
|   └─ pedido/
```

```
|   ├── pedido.go
|   └── pedido_test.go
|
|── .gitignore
|
|── go.mod
|
└── README.md
```

Link de acceso al repositorio en Github:

<https://github.com/JCcoconut/sistema-ecommerce-go>

Conclusiones

El enfoque funcional aporta una ventaja que vale rescatar: una validación fallida no necesita reparar estado parcialmente modificado.

Retornar nuevos valores, manejar errores explícitos y recibir funciones como políticas hace que el comportamiento sea más predecible y comprobable. Al mismo tiempo, la interfaz de consola mantiene la propuesta suficientemente sencilla para una primera unidad y deja una base clara para evolucionar hacia persistencia, servicios web y seguridad.

El sistema de gestión de e-commerce permite integrar los fundamentos iniciales de Go dentro de un proyecto coherente y modular. Las funciones delimitan responsabilidades, los paquetes facilitan la organización del código y las estructuras de datos representan los productos, el carrito, las promociones y los pedidos.

El prototipo constituye una base que posteriormente puede evolucionar hacia una aplicación web con autenticación, seguridad, facturación y pagos electrónicos.